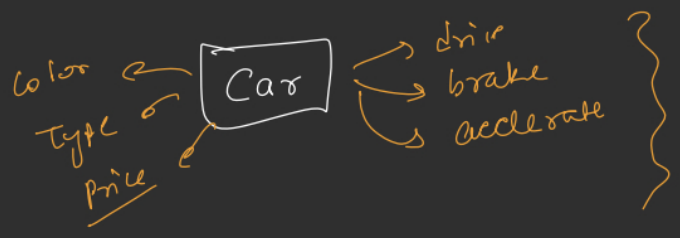
Abstraction

- ??
- The process of focusing on what something does, while ignoring how it does that.



Human → Engineer
Engineer → physics
Physicist → Mathematician

ATM Machine

- deposit
- withdraw
- check Balance

Behaviours

Methods

OOP ⇒ Real world (X)

Humans think about Reality

Teacher → teaches → Class Teacher {
name;
age;
gender;
teach();
takeAttendance();
}

Car → drives

Button → click

Abstraction (Car)

- ① Represent whatever necessary.
- ② Even what we model, we dont want everyone to know how everything works, but they can still use it.

Java

- ① low level Abstraction (Hiding Implementation Details)
- ② High level Abstraction (Seprate WHAT from HOW)

```
Class Car {
    String type;
    start() {
    }
    accelerate() {
    }
    brake() {
    }
    :
}
```

Concrete Classes

```
main() {
    Car c = new Car();
    c.start();
    c.accelerate();
    c.brake();
}
```

High level Abstraction : (Seprate WHAT from HOW)

↳ Abstract classes

↳ Interfaces

```
class FuelCar {
    drive() {}   // 1
}

class ElectricCar {
    drive() {}   // 2
}
```

```
main() {
    ElectricCar ec = new EC();
    FuelCar fc = new FuelCar();
    ec.drive();
    fc.drive();
}
```

```
class Car {
    applyHandbrake() {}
}

class FC extends Car {
    drive() {}
}

class EC extends Car {
    drive() {}
}

Car c = new EC();
        FC();
```

Abstract Class

```
abstract class Car {
    start() {-}
    abstract accelerate();
    abstract brake();
}
```

```
class EC extends Car {
    accelerate() {
        =
    }
    brake() {}
}
```

```
class FuelCar extends Car {
    accelerate() {
        =
    }
    brake() {-}
}
```

Interfaces

- abstract class
- contract

Class → Blue Print of Object

Interface ↛ Object

```
interface Car {
    void start();
    void accelerate();
    void brake();
}
```

```
class ElectricCar implement Car {
    =
    =
    =
}
```

Car c = new ——

```
class FuelCar implements Car {
    strA() {}
    acc() {}
    brake() {}
}
```

Interface

↳ Pure WHAT ?
(Pure Abstraction)

Contract

Intention

Abstract Class

↳ Partial WHAT & How ?

Interfaces

↳ Responsibilities / Capabilities / Roles

interface Car
↳ start
↳ accelerate
↳ brakes

Electric Car

Fuel Car

Comparable
Serializable

ICar

Flyable
Walkable
Runnable

interface Flyable
fly();

class Birds
fly() { }

class Aeroplane
fly() { }

Abstract Class

↳ Families of similar Object

Car

EC

FC

Multiple Inheritance

- Class ✗
- Interface ✓

Abstraction

↓

Data/ Implementation Hiding

Encapsulation

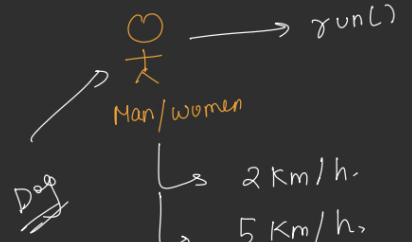
- ① $\{ = \}$
- ② Data-Security
  - ↳ Access Modifiers

Polymorphism

Many Forms ⟹ Real life objects ⇓ They have many forms.

Polymorphism

(1)



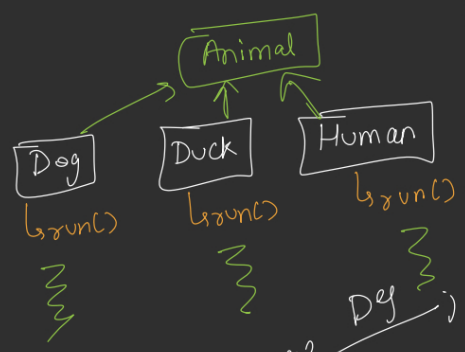
Same Obj. behaves / response differently on same command if one param. is changed.

(compile-time polymorphism)

Method Overloading

```
class Human {
    void run() {
        → 2 Km/h.
    }
    void run(boolean isDog, boolean isHuman) {
        if(true) {
            → 5 Km/h
        }
    }
}
```

(2)




```
Animal a = new Dog();
a.run();
```

Run-time polymorphism

Method Overriding

```
abstract class Animal {
    abstract void run();
}

class Dog extends Animal {
    run() {→}
}
```

Duck: run()

Human: run()

```
Animal a;
  if( type == 'duck')
      a = new Duck();
  else if( ⟶
      a = new Human();
```

Polymorphism

- Compile-time (Method Overloading)
- Run-time (Method Overriding)

Static, Final, Private

Static

↳ override ✗

```
Class A {
    static void fun() {}
}

Class B extends A {
    static void fun() {?};
}

A a = new B();
```